



UiT

THE ARCTIC
UNIVERSITY
OF NORWAY

Faculty of Science and Technology
Department of Computer Science

GameLab: Mythos Grant

Jens Christian Valen Leynse

INF-3910 Computer Science seminar: GameLab - June 2025



Table of contents

Milestone 1: High concept	3
Milestone 2: Design	6
Milestone 3: Prototype	Error! Bookmark not defined.
Milestone 4: Alpha	16
Milestone 5: Beta	18
Milestone 6: Live	20

List of figures

Figure 1. Example figure – Scene transition	Error! Bookmark not defined.
---	-------------------------------------

Milestone 1: High concept

Working title
Mythos Grant

Group names and roles
Jens: Developer, designer

The «X»

When scholarly debates hit a dead end, rival historians armed with legendary sword relics settle the argument in a high-stakes tournament—where only one field of research wins the coveted grant money.

Time/Schedule

Milestone	Task	Description	Duration	Start date	End date
Design	Develop design doc	Game and technology defined	3.5 weeks	01.02.2025	25.02.2025
Production	Program core mechanics	Movement, combat actions, special abilities	3 weeks	26.02.2025	18.03.2025
Prototype	Create prototype	Playable game where characters can move, attack the opponent, and determine a winner or loser.			
Production	Create assets	Characters, background, UI, level design, sound & music	8 weeks	19.03.2025	22.04.2025
Alpha	Alpha testing & bug fixing	Feature complete			
Beta	Balancing	Content complete	4 weeks	23.04.2025	20.05.2025
Launch	Final polish		2 weeks	21.05.2025	04.06.2025

Game genre and theme

Mythos grant is a fast-paced, 1v1 fighting game that blends dynamic platforming with strategic sword combat. The theme is inspired by an academic setting mixed with mythological stories. The stages and music will reflect this theme, blending modern and ancient aesthetics. The soundtrack will combine traditional instruments with electronic beats.

Game Mechanics, player goal, features

Players control characters who can move, jump, and dash around the screen while wielding large (proportionally sized) swords to attack their opponent. Each player has a health bar; when it is depleted, they lose the game. The goal of each player is to deplete the opponent's health before their own. The swords will be able to slash (rotation), stab (direct attack) and parry (timed swing or button push). Each sword user will also have a special ability that affects how they approach their opponent. The abilities are mostly statistical through buffs or debuffs. Each character will therefore have different attack, movement and defensive stats. Examples of abilities include slowing the opponent on a successful hit, extending the duration of a hitbox, and healing upon attacking an opponent. The game will feature local multiplayer, requiring players to either share a keyboard (using separate key bindings) or use an external controller.

Key technical elements

The game will be programmed in Godot v4.3, with art, sound, and music either AI-generated or sourced from online assets.

Art style

The game features a cartoonish art style. Characters have a distinctively limbless design, with their head, hands, and feet invisibly linked to their torso (inspired by Rayman). This design influences their movement and interactions with opponents' swords. Attire is inspired by the nationality of the sword each character uses. Stages—including backgrounds, walls, and platforms—are inspired by academic settings such as libraries, science labs, courtyards, cafeterias, PC labs, and auditoriums.

Story (World/Environment/Characters)

Players take control of characters armed with legendary sword relics, each imbued with unique abilities that reflect their historical significance.

Story: Historians reach an impasse over which ancient myth merits further research—and when scholarly debate falls short, there's only one way to settle it. In a one-off, high-stakes tournament, rival historians armed with legendary sword relics duel for supremacy. The winner's field of study secures the coveted grant money, ensuring their myth will be explored, and celebrated.

Environment: The arenas blend classical academia with mythic battlegrounds, ranging from ancient temples and historic cities to various rooms across a modern campus.

Character: Each character represents a different mythos, bringing their own research, pride, and unique sword-fighting techniques into battle.

Market positioning, target Platform, revenue and distribution model

Mythos Grant is positioned as a competitive 1v1 fighting game with fast, fluid movement inspired by Super Smash Bros. Ultimate, Super Smash Bros. Melee, and Celeste. It targets players who enjoy technical gameplay, precise movement, and strategic combat, offering a fresh twist by introducing mythological themes and unique interactions.

Platform: The game will initially be developed for PC (Windows) with local multiplayer support. Controller compatibility is a secondary priority but may be implemented to provide a familiar experience for fighting game enthusiasts.

Distribution: The plan is to release the game as a downloadable title on itch.io, with potential expansion to be determined.

Concept sketch

Milestone 2: Design

[This chapter should be approximately 5-10 pages]

Design decisions

[Describe design decisions. Is it one player/multiplayer. Is 2D/3D. Data storage, local/global. Etc.]

The game will be a "couch co-op" multiplayer game, allowing players to compete against each other on the same server or PC. While online multiplayer is a potential future feature, the primary focus is on local multiplayer to ensure optimal performance and minimal latency, which is crucial for the fast-paced gameplay.

Game Type:

Multiplayer: Primarily local ("couch co-op") with potential for online multiplayer in future updates.

Single-Player: Not a focus, but a training/ test mode could be added later.

Visual Style:

The game will be 2D or 2.5D, depending on the character models.

The art style will be low-poly, with animations created either through manual frame-by-frame animation or 3D rigging for 2D characters (2.5D).

Data Storage:

Data will be stored locally to minimize loading times and avoid lag spikes. Pre-loading assets will ensure smooth gameplay.

Platform:

Initially designed for PC, but with potential for console compatibility to broaden the player base.

User Interface (UI):

The UI will be minimalist, using simple image layering to maintain the game's aesthetic.

Features

[Describe your features in more details, make priorities]

The game's features are divided into must-have, should-have, and nice-to-have categories to prioritize development.

Core Gameplay Mechanics (Must-Have):

Movement:

Players can jump higher than the average platform height and dash in all 8 directions.

Dashing can transfer momentum when interacting with platforms or walls, adding depth to movement.

A single dash is available until the player lands or performs a wall jump.

There will be one button for both jump and dash.

Combat:

Players can swing or focus their sword.

The sword follows the player, either trailing behind or leading in front, depending on whether it's focused.

Sword swings gain power based on distance traveled but follow the shortest path to the target.

Parrying is possible by focusing the sword in front of the player. Successful parries stun the opponent, but parrying a swing causes the defender to lose to power.

Rock-Paper-Scissors Mechanics:

Stab beats Swing.

Parry beats Stab.

Swing beats Parry.

Special Abilities:

Each sword has a unique ability, such as:

Harpe: Slows the opponent's movement.

Tyrfing: Grants additional attack power but causes exhaustion afterward.

Nuada: Heals the user after multiple successful hits.

Zulfiqar: Performs a sweeping attack that can block enemy attacks.

Lævateinn: Confuses the opponent, randomizing their first three dashes.

Exploration and Progression (Should-Have):

Stages:

Each stage will have a unique background and platform setup.

The winner of each match influences the next stage, adding a dynamic element to gameplay.

Nice-to-have features include thematic elements (e.g., mythological references) and visual storytelling in the background.

Character Customization:

Players can earn in-game currency to unlock armor for their characters.

After 5 victories, a player's character becomes fully armored, symbolizing their dominance.

Accessibility and Difficulty (Must-Have):

The game's difficulty is player-driven, with no AI opponents or obstacles.

Character silhouettes will be distinct to ensure visibility, even for players with visual impairments.

Voice Lines (Nice-to-Have):

Characters will have voice lines when entering the stage, adding personality and immersion.

Example: A character might say,

"Let's make history—by erasing yours."

"I'd cite my sources, but you won't live to read them."

"Your thesis? Flawed. Your methods? Laughable. Your fate? Sealed."

"This blade has seen centuries. You won't last minutes."

"They say 'sword name' is unbreakable. Shame you're not."

Asset list

[Which type of assets will you use (sound, graphics, other). Will you create or buy them, AI?.]

The game will require a variety of assets, including graphics, sound, and UI elements.

Graphics:

Character Models: Low-poly designs, either 2D or 2.5D.

Backgrounds: Unique environments for each stage.

Platforms and Walls: Customizable designs to match the stage themes.

UI Elements: Health bars, menus, and buttons.

Weapon Models: Unique designs for each sword.

Sound:

Background Music: Upbeat electronic music created using AI tools.

Sound Effects: Sword clashes, player damage, dashing, jumping, and stage transitions.

Voice Lines: Short phrases for character entrances and victories.

Other Assets:

Fonts: Easy-to-read, minimalist fonts.

Particle Effects: Sparks for sword clashes, dust for jumps and dashes.

Workflow:

Core Mechanics:

Default character, sword, stage, and platform models.

Multiplayer Integration:

Implement local multiplayer functionality

Immersion Mechanics:

Health UI, music, sound effects, and particle effects.

Additional Mechanics:

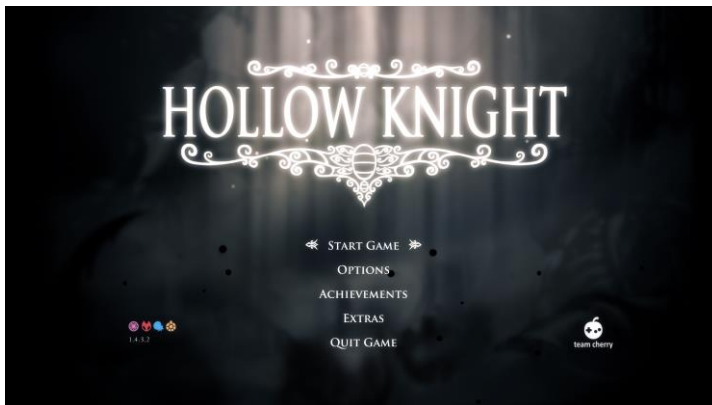
Menu screens, 8 sword models, 3 stages, and 8 wall designs.

Visual overflow

[Describe the different scenes. Include figures of all/most important screen.]

Scene Descriptions:

Main Menu: A simple, intuitive interface with options for Play, Controls, Settings, and Credits.



Like this only with thematic elements corresponding to the myths featured in the game.

Gameplay: Fast-paced combat on dynamic stages with distinct themes.



I want the platform layout to be inspired by *Celeste*'s level design, where the foreground and background blend seamlessly to create a cohesive feel. However, the stages need to be much more open. I imagine some platforms and walls in the middle to break up the space and add more variation.

Victory Screen: A celebratory screen showcasing the winning player's character.
To be decided

Select Screen: Players choose their sword, stage, and music.



I want the character select screen to resemble this, but instead of characters, players will select swords (which correspond to characters). Both the swords and controls will be displayed on the sides, while common options like stage and music will be centered.

Control Screen: Customize controls for keyboard and controller.
To be decided

Settings: Adjust screen mode, music, and sound settings.
To be decided

User Flow:

Players start at the Main Menu.

They select Play and choose their characters and stage (music is set to default unless changed).

After the match, they return to the Main Menu or proceed to the next stage.

Art Style:

The low-poly aesthetic ensures a clean, modern look while keeping performance optimal.

Animations:

Character Movements: Smooth transitions for jumps, dashes, and attacks.

Environmental Effects: Subtle animations like moving clouds or flickering lights in the background.

Revenue plan

[How will you make money from the game?]

Monetization Strategies:

Self-Publishing: Release the game at a low cost and focus on marketing through social media and influencers.

Publisher Partnership: Pitch the game to a publisher for additional funding.

Post-Launch Content: Update with new stages, characters, or swords.

Pricing Model:

The game will be free at launch for alpha users to provide feedback, then increased to an adaptable price based on interest.

Marketing Plan:

Social media: Promote the game on platforms like Twitter, Instagram, and TikTok.

Influencers: Partner with gaming YouTubers and Twitch streamers.

Scope and time estimate

[More detailed description of how you plan to complete the game, including time need for training, creating assets, coding]

Development Phases:

Training

- Task: Learn new tools (e.g., Blender, Godot).
 - Duration: 2 weeks (integrated into Production phase).
 - Start Date: 26.02.2025.
 - End Date: 11.03.2025.

Asset Creation

- Characters and Swords:
 - Duration: 1 month (integrated into Production phase).
 - Start Date: 19.03.2025.

- End Date: 18.04.2025.
 - Stages and Backgrounds:
 - Duration: 1.5 months (integrated into Production phase).
 - Start Date: 19.03.2025.
 - End Date: 22.04.2025.
 - UI and Sound:
 - Duration: 1 month (integrated into Production phase).
 - Start Date: 19.03.2025.
 - End Date: 22.04.2025.
-

Coding

- Core Mechanics:
 - Duration: 2 months (integrated into Production phase).
 - Start Date: 26.02.2025.
 - End Date: 22.04.2025.
- Additional Features:
 - Duration: 1.5 months (integrated into Production phase).
 - Start Date: 26.02.2025.
 - End Date: 22.04.2025.

Technical design

[Are there any technical requirements, e.g., high end hardware, specific game engine, etc.]

Game Engine:

Godot: Chosen for its flexibility, cross-platform support, and ease of use.

Software Tools:

Blender: For 3D modeling.

GIMP: For textures and UI design.

Audacity: For sound editing.

ComfyUI/Fugatto: For AI-generated assets.

Performance Optimization:

Reduce polygon counts for models.

Optimize code to minimize resource usage.

Risks

[Are there any risks in your project, e.g., what may cause you to run behind schedule]

Technical Risks:

Performance bottlenecks on lower-end hardware.

Networking issues for online multiplayer (if added).

Schedule Risks:

Delays in asset creation or coding.

Budget Risks:

Unexpected costs for tools or assets.

Market Risks:

Competition from similar games.

Changing player preferences.

Mitigation Strategies:

Regular testing to catch issues early.

Flexible timelines to accommodate delays.

Contingency budget for unexpected expenses.

Milestone 3: Prototype

Prototype implementation

Core Mechanics Implemented

- **Movement:** Characters can move left/right, jump, and dash in 8 directions. Dashing transfers momentum when interacting with the ground
- **Combat:** Basic sword swings are implemented.
- **Multiplayer:** Local multiplayer functionality is implemented, allowing two players to share a keyboard or use a controller and a keyboard.

Assets Used

- **Characters:** Default low-poly models with limbless designs.
- **Swords:** Basic sword model for testing.
- **Stage:** A single test stage used from Smash ultimate's stage design.
- **UI:** State and frame information for testing

Technical Implementation

- **Engine:** Godot v4.3 is used for development.
- **Physics:** Custom physics for movement, dashing, and sword interactions.
- **Input Handling:** Keyboard and controller inputs are mapped for multiplayer compatibility.

Prototype goals

The primary goals of the prototype were:

1. **Core Gameplay:** Implement basic movement, combat, and multiplayer functionality.
2. **Technical Feasibility:** Test the viability of the game's mechanics in Godot.
3. **Player Interaction:** Ensure smooth and responsive controls for local multiplayer.
4. **Art Style Validation:** Confirm that the low-poly, limbless character design works in a 2D environment.
5. **A win condition:** To be able to play an early rendition of my game with close to intended core mechanics.

Game state

Current State:

- The prototype is playable with basic movement, combat, and multiplayer functionality.
- Placeholder assets are used for swords, and stages.
- Core mechanics like dashing, jumping are functional.
- Sword interactions are in the works.

Missing Features:

- **Combat Mechanics:** Stabbing, parrying, and special abilities are not yet implemented.
- **Stages:** Only one test stage exists; additional stages with unique themes and platform layouts are needed.
- **UI/UX:** Menus and victory screens with thematic elements.
- **Sound and Music:** No sound effects or background music are included.
- **Variable Tweaks:** Balancing movement speed, dash distance, and attack power is pending.

What did I learn

Technical Insights:

- Godot's physics engine is well-suited for 2D platforming and combat mechanics.
- Implementing multiplayer functionality locally is straightforward but requires careful input mapping.

Design Challenges:

- Balancing movement and combat mechanics is crucial for maintaining fast-paced gameplay.
- Pixel art requires creative animation solutions with time consuming but rewarding results.

Next Steps:

- Focus on implementing stabbing, parrying, and special abilities to complete the combat system.
- Add more stages with unique themes and platform layouts.
- Create the UI/UX and integrate sound effects and music.

Prototype Progress Summary		
Feature	Status	Notes
Movement (Jump, Dash)	Implemented	Momentum transfer during dashes needs fine-tuning.
Basic Sword Swings	In the works	Sword rotation needs fixing.
Multiplayer (Local)	Implemented	Keyboard and controller inputs are mapped.
Stabbing and Parrying	Not Implemented	Core combat mechanics still in development.
Special Abilities	Not Implemented	Planned for future updates.
Stages	1 Test Stage	Additional stages with unique themes needed.
UI/UX	Not Implemented	Requires thematic polish and additional features (e.g., settings menu).
Sound and Music	Not Implemented	Background music and sound effects are pending.

Milestone 4: Alpha

[Chapter 3-6 should combined be around 10-15 pages.]

Alpha Implementation

For the alpha deadline, I aimed to get the foundational combat mechanics working so I could begin evaluating character responsiveness and interaction. I chose to work from the inside out, starting with player physics and direct interactions before implementing surrounding systems like health, menus, or sound. The primary focus was establishing the movement system, weapon mechanics, and hit reactions.

Players can be controlled either via keyboard or controller. Movement is driven by a physics-based system implemented through code, where forces and velocities determine how characters behave rather than relying on animation-driven motion. I began testing the combat loop using body collisions between the two characters, as sword mechanics had not yet been implemented at that point. Once swords were introduced, the attack logic shifted to apply knockback forces depending on a damage value that increases with each successful strike. To make impacts more dynamic and directional, I tuned the knockback angle based on the relative horizontal position between the attacker and the defender, which gave attacks a more intuitive and satisfying feel.

To enhance the visual weight and responsiveness of combat, I implemented both hitlag and hitfreeze. On impact, the game briefly pauses movement to emphasize the moment of contact, improving both readability and game feel. The sword itself is designed as a pendulum that trails the player's movements. It has a constantly active hitbox that rotates around the character, serving as both a weapon and a visual cue. An additional attack trigger was implemented that temporarily increases the angular momentum of the sword, allowing players to deliberately swing it toward the opponent. This made attacks feel more intentional and expressive, giving players some directional control over their offense.

Refining the pendulum mechanic took significantly more time than expected. Achieving a balance between responsiveness and smoothness required tuning trailing delay, damping values, and pivot responsiveness. A major issue arose with physics interpolation; at higher movement speeds, the sword would spin erratically or fail to track correctly. To resolve this, I moved away from Godot's built-in interpolation and wrote a custom method that tracked movement more consistently and preserved the feel of momentum.

Alpha Goals

My primary objective was to create a working prototype of a fight between two characters, focusing exclusively on the mechanics of movement and attack. I wanted to see whether the concept of a pendulum-swinging sword would support a functional and interesting game loop. This goal was met. The two characters are now capable of basic movement and interaction, with visible reactions to being hit, working directional knockback, and functional inputs via both controller and keyboard. While not yet a full game, the foundation is in place.

By the end of the alpha phase, the core systems I had working included character movement with dash integration and variable jump height, pendulum-based sword mechanics with dynamic hitbox generation, damage-scaling knockback, hitlag and hitfreeze for visual

emphasis, and the ability for two characters to be played simultaneously with different input devices.

Game State

At this point, the project exists as a functional physics prototype. Characters can move, jump, and attack, and there is a rudimentary sense of combat through knockback and sword swings. Some player feedback is included via the hit pause effect and visual movement of the sword. The player experience is limited to these mechanical systems, and the game lacks any metagame structure or long-term feedback systems.

The following features are missing at this stage:

There is no user interface for menus, match flow, or player instructions. No health system has been implemented, so there is no way to determine victory or defeat. Visual polish such as character effects, a background image, or screen transitions are absent. The game has no sound or music. While the dash movement is in place, wall jumping and the associated traversal mechanics have not yet been added. Particles and polish-level feedback effects are also not yet implemented.

During this phase, I replaced the original parry mechanic with a system where the player knocks away the opponent's sword using momentum. Controller support was relatively simple to add, due to the modular input system; I only needed to add buttons that mapped controller input to the same actions. Although I had built the stage, a background had not yet been added—it will be completed in the Beta phase. At this point, most of my development approach was to “make it functional,” so minor bug fixing and code optimization were deferred for later.

What Did I Learn

This phase of the project underscored how important it is to iterate early and often, especially when dealing with physics-based movement and unconventional weapon mechanics. I learned how subtle changes to timing, thresholds, and physics values can drastically alter how the game feels. Even without traditional animation, the game could convey a lot through timing and responsive physics reactions.

Getting the sword mechanic to behave consistently was especially instructive. It forced me to move beyond the tools provided by the engine and develop a deeper understanding of motion tracking, interpolation, and input mapping. The process also revealed how much debugging is required even for seemingly simple systems—like knockback angles or attack detection—when they are part of a real-time physics environment.

Finally, regular playtesting was essential. Watching how the character responded to hits or how the sword moved under different circumstances gave me clear and immediate feedback about what needed to be adjusted. The tight loop of test, tweak, and test again was invaluable in shaping the feel of the game at this early stage.

Milestone 5: Beta

[Chapter 3-6 should combined be around 10-15 pages.]

Beta implementation

For the beta deadline, I extended and refined the systems built during Alpha and added polish to move the experience closer to a complete game. One major addition was a working health system. I had already incremented a hidden damage value when players were hit, but I now implemented a visual health bar with segments that change color as health decreases. This health value also feeds back into knockback calculations, and when a player's health reaches zero the round ends (with continuation logic to be added later). The health value also affects knockback, however this either needs to be dampened or replaced with a static value, as it currently is a bit too powerful.

I created a functioning menu system with scene transitions, allowing the player to go from the main menu to gameplay. I encountered some challenges with controller navigation focus. Ensuring a controller D-pad and keyboard buttons could reliably navigate between options required adjusting focus modes and rewriting parts of the button interaction logic.

The audio was layered in next. Background music and impact sound effects are driven by `AudioStreamPlayer` nodes. I balanced volumes to avoid pitch spikes, ensured music levels never drowned out effects, and triggered distinct attack, hit, jump, and landing sounds to enhance feedback.

On the movement side, wall jumping was implemented alongside a more forgiving dash-input buffer, letting players queue a jump or dash slightly beyond the end of the initial dash state. This smoothing of input timing makes advanced maneuvers more accessible.

Finally, several visual-feedback features were added: screen shake now triggers on player hits, and a brief color-flash indicates successful contacts. Together these effects give combat a more satisfying snap and responsiveness.

Beta Goals

The aim for Beta was to solidify the core combat loop with health, feedback, and basic UI flow, transforming the alpha into something that feels like a cohesive product. Although the win condition is still in progress, there's only small additions that need to be made for it to be a coherent flow. I sought to refine player-input timing, and layer in audiovisual feedback so that each action—dash, jump, slash, hit—carries appropriate weight and clarity.

Game State

At the conclusion of Beta, the project includes a complete combat loop: movement with dashing, jumping, and wall jumping; pendulum-based sword attacks; health bars tied to victory logic; responsive knockback; hit pause and hitflash; screen shake; and layered sound and music. A navigable main menu leads into gameplay, and basic scene transitions are in place. I added a background image to enhance the map.

Remaining work includes a full character/sword select screen, options toggles for sound and music, victory-screen visuals, polished particle effects, and continued art/sound asset passes. Once these are added, the game will be ready to move into final polishing and launch

preparations. I also have not implemented animations yet; they might get postponed until after the game goes live.

Milestone 6: Live

[Chapter 3-6 should combined be around 10-15 pages.]

Alpha implementation

Alpha goals

Game evaluation/game state

Future work

Cost breakdown

What did I learn?

Screenshots

Student contribution

[Detailed description of how each group member contributed to the project. Please inform TAs if there are problems with group dynamics and student contributions.]
